***Володимирський педагогічний фаховий коледж імені Агатангела Кримського***
циклова комісія викладачів професійної цифрової та математичної освіти

**Інструкція до практичної роботи №61**

**ТЕМА: Робота з растровими зображеннями та іконками за допомогою CTkImage та бібліотеки Pillow (PIL).**

*Освітньо-професійна програма: Професійна освіта (Цифрові технології)*
*Тривалість заняття: 80 хв*

**Мета заняття:**

- ознайомитися з відмінністю між векторною та растровою графікою;
- вивчити клас CTkImage для показу растрових зображень у CustomTkinter;
- ознайомитися з бібліотекою Pillow (PIL) для завантаження та обробки зображень;
- навчитися виводити зображення на віджетах і на полотні CTkCanvas.

## Теоретичні відомості

### *1. Векторна графіка проти растрової*

Усі малюнки, побудовані в попередніх практичних роботах методами create_rectangle(), create_oval() тощо, є векторною графікою: фігура описується математично (координатами, розмірами), а не набором окремих точок-пікселів. Растрова графіка — це, навпаки, зображення, збережене як прямокутна сітка пікселів, кожен з яких має власний колір (саме так влаштовані фотографії й більшість файлів .png, .jpg). Растрові зображення не можна «намалювати» примітивами полотна — їх потрібно завантажити з готового файлу.

### *2. Бібліотека Pillow (PIL)*

Pillow — це стандартна, найпоширеніша бібліотека Python для роботи із зображеннями: завантаження файлів різних форматів, зміна розміру, обрізка, малювання на зображенні, застосування фільтрів. Історично бібліотека називалася PIL (Python Imaging Library), тому в коді її підключають під іменем PIL, хоча встановлюють як пакет pillow:

```
pip install pillow

from PIL import Image

img = Image.open('photo.png')
print(img.size)   # (ширина, висота) у пікселях
```

### *3. Клас CTkImage*

CustomTkinter не показує зображення Pillow напряму — спочатку їх потрібно обгорнути класом CTkImage, який автоматично адаптує зображення до поточного режиму оформлення (можна навіть задати різні варіанти зображення

для темної й світлої теми одночасно) та коректно масштабує його на екранах із різною роздільною здатністю:

```
from PIL import Image
import customtkinter as ctk

pil_image = Image.open('icon.png')
ctk_image = ctk.CTkImage(light_image=pil_image,
dark_image=pil_image, size=(100, 100))
```

Параметри light_image і dark_image дозволяють використати різні файли зображень для світлої й темної теми (наприклад, лого з темним текстом для світлого фону і зі світлим текстом — для темного); якщо зображення однакове для обох тем, обидва параметри просто вказують на той самий об'єкт. Параметр size задає розмір показу у форматі (ширина, висота), незалежно від фактичного розміру вихідного файлу.

### *4. Показ зображення в CTkLabel*

Найпростіший спосіб показати зображення — передати об'єкт CTkImage параметру image мітки CTkLabel (текст при цьому зазвичай залишають порожнім, якщо потрібне лише зображення):

```
label = ctk.CTkLabel(app, image=ctk_image, text='')
label.pack(pady=20)
```

### *5. Створення зображення програмно засобами Pillow*

Pillow дозволяє не лише завантажувати готові файли, а й створювати зображення «з нуля» програмно, малюючи на них через модуль ImageDraw, — це корисно для генерації простих значків без потреби створювати окремі файли зображень:

```
from PIL import Image, ImageDraw

img = Image.new('RGBA', (150, 150), (0, 0, 0, 0))   # прозоре тло
draw = ImageDraw.Draw(img)
draw.ellipse((10, 10, 140, 140), fill=(58, 126, 191, 255))
draw.ellipse((45, 45, 105, 105), fill=(255, 255, 255, 255))

ctk_img = ctk.CTkImage(light_image=img, dark_image=img, size=(120,
120))
ctk.CTkLabel(app, image=ctk_img, text='').pack(pady=10)
```

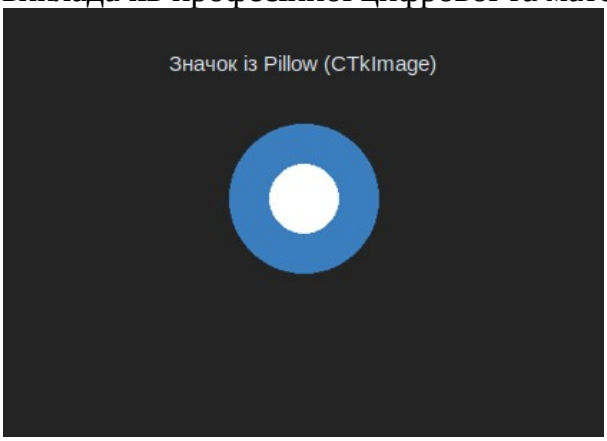



*Рис. 1. Значок, повністю згенерований програмно засобами Pillow (ImageDraw) і показаний через CTkImage*

Формат кольору 'RGBA' у Image.new() означає, що кожен піксель зображення описується чотирма числами: червоний, зелений, синій канал (від 0 до 255) та альфа-канал прозорості (0 — повністю прозоро, 255 — повністю непрозоро). Саме альфа-канал дозволяє створювати значки з прозорим тлом, які органічно вписуються в будь-яке оформлення застосунку.

### *6. Показ зображення на полотні CTkCanvas*

Растрове зображення можна розмістити й безпосередньо на полотні (а не лише в CTkLabel) методом create_image() — це корисно, коли зображення потрібно поєднати з іншими векторними примітивами в одній композиції або з анімацією:

```
from PIL import Image, ImageTk

pil_img = Image.open('sprite.png')
tk_img = ImageTk.PhotoImage(pil_img)
canvas.create_image(100, 100, image=tk_img)
```

Зверніть увагу: для розміщення зображення саме на полотні (а не у звичайному віджеті) використовують клас ImageTk.PhotoImage зі стандартного tkinter, а не CTkImage — це технічна особливість Canvas, який працює безпосередньо із зображеннями нижчого рівня.

## Хід роботи

### *Варіант і вимоги до здачі*

a) N — ваш порядковий номер у списку групи (журнал). Розмір зображення і діаметр круга залежать від N (mod — остача від ділення):

| Завдання | Дані варіанта | Приклад N = 7 |
|---|---|---|
| 2 | розмір S = 100 + 10·(N mod 5) | 120×120 |
| 4 | діаметр d = 80 + 10·(N mod 3); колір за N mod 3 = 0, 1, 2: #e74c3c, #3498db, #2ecc71 | d = 90, #3498db |

б) Кожне завдання — окремий файл Прізвище_ПР61_зN.py (наприклад, Ivanenko_PR61_z3.py), код текстом, а не фото чи скріншотом. Файли з прикладів із теорії та додаткові файли, яких немає в умові, здавати не потрібно.

в) Запускайте програми в IDLE або іншому середовищі на своєму комп'ютері; до звіту (документ Word) додайте скріншот виконання кожної програми.

г) Для кожного завдання додайте скріншот вікна із видимим заголовком. Імпорт — import customtkinter as ctk; програма завершується app.mainloop(). Тексти елементів і повідомлень — точно як в умові. Зображення зберігайте посиланням на об'єкт CTkImage (щоб його не прибрав збирач сміття); файли зображень здавайте разом із програмою.

д) У коді залиште короткі коментарі власними словами перед основними блоками.

е) Будьте готові пояснити будь-який рядок свого коду й внести в нього невелику зміну (див. розділ «Захист роботи»).

1. Встановіть бібліотеку Pillow командою pip install pillow (якщо ще не встановлена) і додайте до звіту скріншот результату команди.

   **Результат:** у звіті є скріншот зі встановленою Pillow.

2. Створіть файл Прізвище_ПР61_з2.py. Завантажте довільний .png (Image.open) і покажіть його в CTkLabel через CTkImage розміру S×S вашого варіанта.

   **Приклад (N = 7):** size=(120, 120); файл зображення здайте разом із програмою.

3. Змініть size зображення на три значення (наприклад, 60, 120, 200) і додайте до звіту три скріншоти з підписами.

   **Приклад:** під кожним скріншотом вказано розмір.

4. Створіть засобами ImageDraw зображення RGBA із прозорим тлом і кольоровим кругом діаметра d вашого варіанта, покажіть його через CTkImage.

   **Приклад (N = 7):** d = 90, колір #3498db; поза кругом — прозорий фон.

5. Розмістіть зображення на CTkCanvas (create_image) разом із текстом і лінією в одній композиції.

**Приклад:** зображення, текст і лінія не накладаються.

6. Завдання підвищеної складності*. Напишіть функцію make_dot(color, d) на ImageDraw, що повертає зображення кола заданого кольору; за її допомогою покажіть три індикатори («світлофор»: зелений, жовтий, червоний) поруч із проміжком 20 пікселів.

   **Приклад:** порядок зліва направо: зелений, жовтий, червоний; d = 60.

## Критерії оцінювання

| Складова | Оцінювання | Макс. балів |
|---|---|---|
| Завдання 1–5 | кожне по 1,8 бала | 9 |
| Завдання 6* | 2 бали | 2 |
| Контрольні питання | відповіді у звіті | 1 |
| **Разом** | | **12** |

Кожне завдання оцінюється так: програму виконано повністю, вона працює на вимогах умови й прикладах формату виводу — 100% балів; програма працює, але є суттєве зауваження (наприклад, не встановлено Pillow, зображення зникає (немає посилання на CTkImage), немає прозорості, розмір не відповідає варіанту) — 50%; програма не працює або не здана — 0. Завдання 1–5 оцінюються по 1,8 бала, завдання 6* — 2 бали. Контрольні питання оцінюються в 1 бал, якщо на всі питання дано правильні відповіді.

### *Захист роботи*

Перед виставленням оцінки викладач може попросити вас пояснити будь-який рядок свого коду, показати, як змінюється значення змінних на прикладі, або внести в програму невелику зміну (наприклад, змінити розмір чи колір зображення). Якщо пояснити код не вдається, оцінку за відповідне завдання може бути знижено.

## Контрольні питання

Відповідайте на питання письмово у звіті (документ Word разом із скріншотами): 1–2 речення на кожне питання, власними словами.

1. Чим растрова графіка принципово відрізняється від векторної?
2. Для чого призначена бібліотека Pillow і як вона підключається в коді (під яким іменем)?
3. Навіщо потрібен клас CTkImage, якщо зображення вже завантажене об'єктом Pillow?

4. Що означають чотири канали кольору 'RGBA' і навіщо потрібен альфа-канал?
5. Яким класом і методом зображення показують безпосередньо на полотні CTkCanvas?